SmartBookingPlatform

Microservices Architecture Report

Leonil Sulude

April 7, 2026

Contents

1 Introduction **4**

2 System Architecture **4**
2.1 Architectural Roles . . . 5
2.2 Communication Model . . . 5
2.3 Why Microservices Were Chosen . . . 6
2.4 High-Level Architecture Diagram . . . 6
2.5 Architectural Observations . . . 7

3 Service Design **8**
3.1 Auth Service . . . 8
3.2 Catalog Service . . . 8
3.3 Booking Service . . . 9
3.4 API Gateway . . . 9
3.5 Discovery Service . . . 10

4 Data Architecture and Service Isolation **10**
4.1 Database per Service . . . 10
4.2 Database Schema Overview . . . 10
4.3 Rationale . . . 12
4.4 Data Consistency Across Services . . . 12
4.5 Trade-offs . . . 12
4.6 Schema Management . . . 12
4.7 Future Evolution: Event-Driven Data Consistency . . . 13
4.8 Lessons Learned . . . 13

5 Inter-Service Communication and Resilience **14**
5.1 Service Communication . . . 14
5.2 Failure Scenarios . . . 14
5.3 Resilience Mechanisms . . . 14
5.3.1 Retry . . . 15
5.3.2 Circuit Breaker . . . 15
5.3.3 Time Limiter . . . 15
5.4 Fallback Strategy . . . 16
5.5 Design Decisions . . . 16

5.6 Trade-offs . . . 16
5.7 Future Improvements . . . 17
5.7.1 Kafka and Event-Driven Communication . . . 17
5.7.2 Other Improvements . . . 17

6 Security Architecture **17**
6.1 Authentication Flow . . . 18
6.2 Example Requests . . . 18
6.3 JWT Token Structure . . . 19
6.4 Authorization at the Gateway . . . 19
6.5 JWT Validation Implementation . . . 19
6.6 Token Propagation . . . 20
6.7 Role-Based Authorization . . . 20
6.8 Public vs Protected Endpoints . . . 20
6.9 Key Design Decisions . . . 20
6.10 Lessons Learned . . . 21

7 Testing Strategy **22**
7.1 Overview . . . 22
7.2 Testing Layers . . . 22
7.3 Unit Tests . . . 22
7.3.1 Auth Service . . . 22
7.3.2 Catalog Service . . . 24
7.3.3 Booking Service . . . 25
7.4 Test Summary . . . 26
7.5 Test Gaps and Limitations . . . 27
7.6 Load Testing with k6 . . . 27
7.6.1 Auth Baseline Test . . . 27
7.6.2 Catalog Load Test . . . 28
7.6.3 Booking Flow Test . . . 28
7.7 Lessons Learned . . . 29

8 Architectural Trade-offs **30**
8.1 Microservices vs Monolith (Solo Developer Perspective) . . . 30
8.2 Synchronous vs Asynchronous Communication . . . 30
8.3 Strong vs Eventual Consistency . . . 31
8.4 Centralized Gateway vs Distributed Security . . . 32
8.5 Database per Service vs Shared Database . . . 33

9 Observability and Logging **36**
9.1 Logging Architecture . . . 36
9.1.1 RabbitMQ Configuration . . . 36
9.1.2 Log Event Structure . . . 36
9.1.3 Log Event Consumer . . . 37
9.1.4 Log Storage . . . 37
9.2 Correlation ID Propagation . . . 37
9.2.1 Generation at the Gateway . . . 37
9.2.2 Propagation to Internal Services . . . 38

9.2.3 Known Limitation: MDC and WebFlux . . . 38
9.3 Current Observability Stack . . . 38
9.4 Observability Gaps and V2 Direction . . . 38
9.5 Lessons Learned . . . 39

10 Deployment and Local Infrastructure **39**
10.1 Infrastructure — Docker Compose . . . 39
10.2 Service Ports . . . 40
10.3 Platform Scripts . . . 40
10.3.1 start-platform.sh . . . 40
10.3.2 stop-platform.sh . . . 41
10.4 Startup Observations . . . 41
10.5 Lessons Learned . . . 41

11 Limitations and V2 Roadmap **42**
11.1 Known Limitations . . . 42
11.2 V2 Roadmap . . . 43
11.2.1 Messaging — Kafka . . . 43
11.2.2 Observability — Metrics and Tracing . . . 43
11.2.3 Security — Role-Based Authorization . . . 43
11.2.4 Testing — Integration and Contract Tests . . . 44
11.2.5 Deployment — Kubernetes and Flyway . . . 44
11.2.6 Concurrency — Booking Integrity . . . 44
11.3 V2 Summary . . . 45

12 Conclusion **45**

1 Introduction

Modern backend systems increasingly rely on distributed architectures to support scalability, maintainability, and independent service evolution. Traditional monolithic applications often struggle to scale development teams and infrastructure as systems grow in complexity.

The **SmartBookingPlatform** project was developed to explore the design and implementation of a microservices-based backend architecture using modern Java and Spring technologies. The platform simulates a simplified service marketplace where providers can publish service offers and customers can create bookings.

The system is composed of multiple independent services responsible for different business capabilities. Each service is deployed and executed independently while communicating through REST APIs.

The core architectural goals of the project include:

- Separation of concerns through service decomposition
- Centralized API entry via an API Gateway
- Secure communication using JWT-based authentication
- Dynamic service discovery through Netflix Eureka
- Resilient service-to-service communication
- Load testing and performance evaluation

The platform consists of the following microservices:

- **API Gateway** – central entry point for client requests
- **Auth Service** – user registration and authentication
- **Catalog Service** – management of service offers and resources
- **Booking Service** – booking management and availability validation
- **Discovery Service** – service registry using Netflix Eureka

In addition to the core services, the system includes infrastructure components such as PostgreSQL databases, RabbitMQ messaging, and automated platform startup scripts.

This report documents the architecture, design decisions, communication patterns, resilience mechanisms, and performance evaluation of the platform.

2 System Architecture

The SmartBookingPlatform follows a **microservices architecture**, where the system is decomposed into multiple independent services, each responsible for a specific business capability. This architectural style was chosen to improve separation of concerns, modularity, and service independence.

In contrast to a monolithic architecture, microservices allow each domain area to evolve more independently. This is especially useful when different services have different responsibilities, scaling requirements, or future deployment needs.

The platform is composed of the following core services:

- **API Gateway** – central entry point for all client requests
- **Auth Service** – handles user registration and authentication
- **Catalog Service** – manages service offers and resources
- **Booking Service** – handles booking creation and availability rules
- **Discovery Service** – provides service registration and discovery using Netflix Eureka

2.1 Architectural Roles

Each service in the platform was designed with a clear and isolated responsibility.

Service	Responsibility
API Gateway	Central entry point for all external requests. Responsible for request routing, JWT validation for protected endpoints, and enforcing authentication policies before forwarding traffic to internal services.
Auth Service	Handles user registration and authentication. Generates JWT tokens upon successful login, which are later used by clients to access protected endpoints through the API Gateway.
Catalog Service	Manages the marketplace catalog. An **Offer** represents a service provided by a provider (for example a hotel room or a service appointment). Each offer can contain one or more **Resources**. A resource represents a concrete bookable unit (e.g., a specific room or asset) with attributes such as price, duration, and availability periods.
Booking Service	Responsible for booking creation and validation. Before creating a booking, the service retrieves resource information from the Catalog Service to validate availability, detect scheduling conflicts, and enforce business rules regarding booking periods.
Discovery Service	Provides service discovery using Netflix Eureka. All services register themselves with the discovery server, allowing dynamic resolution of service locations without hardcoded network addresses.

Table 1: Core services and responsibilities

2.2 Communication Model

The communication model is based on **HTTP REST APIs**. Client requests are always sent to the **API Gateway**, which acts as the single entry point into the system.

The gateway is responsible for:

- routing requests to the correct service
- validating JWT tokens for protected endpoints

- centralizing cross-cutting concerns such as authentication and request filtering

Service-to-service communication is performed using **OpenFeign clients**. Instead of using hardcoded URLs, services are resolved dynamically through **Netflix Eureka**, which acts as a service registry.

This means that the Booking Service does not need to know the physical location of the Catalog Service in advance. Instead, it resolves the target instance through service discovery at runtime.

2.3 Why Microservices Were Chosen

A microservices architecture was selected primarily as a learning and design exercise to understand distributed backend systems. This choice makes it possible to study several important architectural concepts in isolation, such as:

- service decomposition
- dynamic routing
- service discovery
- stateless authentication
- inter-service communication
- resilience patterns
- load testing in distributed systems

Although microservices introduce additional complexity compared to a monolith, they also provide a more realistic environment for learning modern backend architecture patterns.

2.4 High-Level Architecture Diagram

Figure 1 illustrates the high-level architecture of the SmartBookingPlatform.

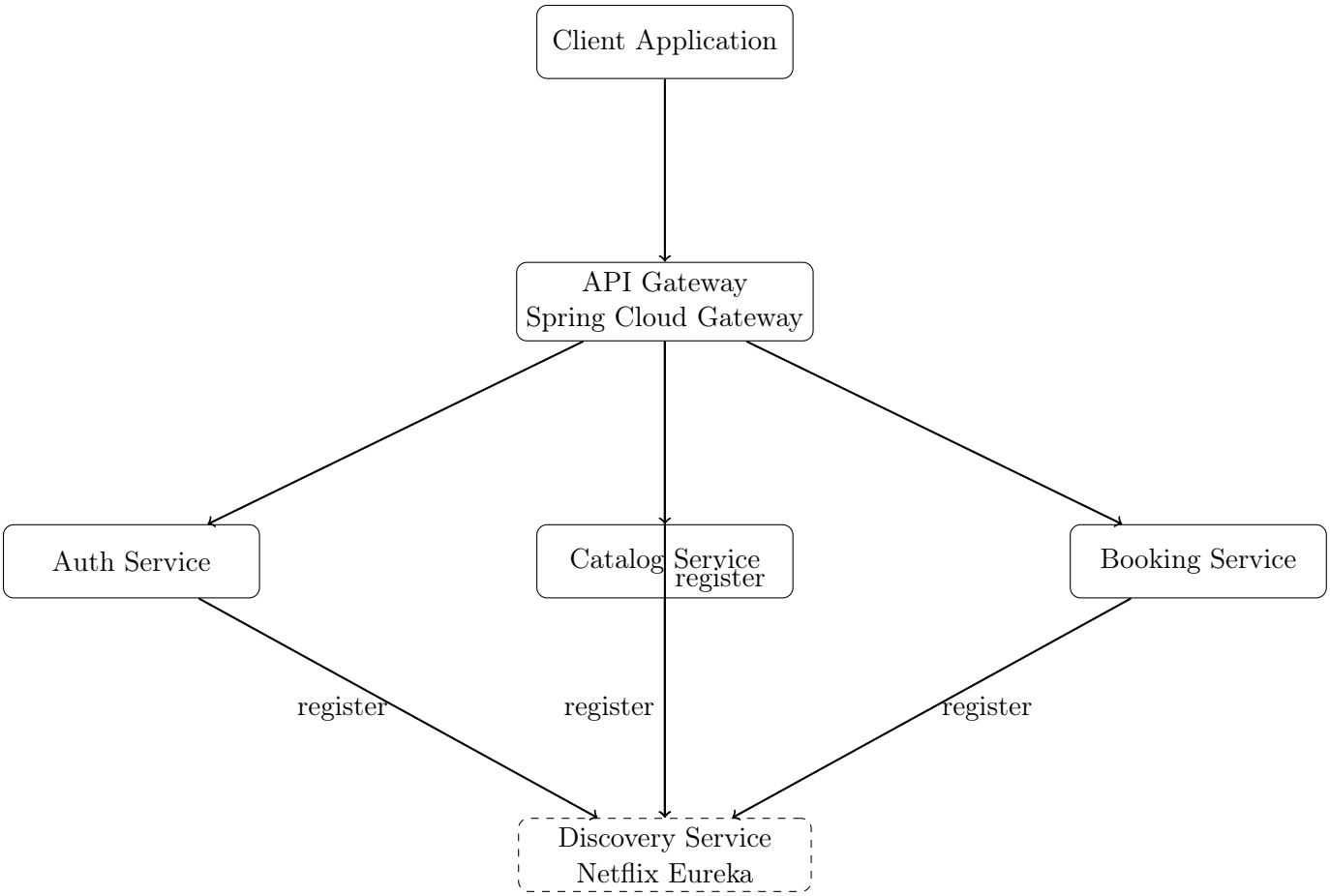

Figure 1: High-level architecture of the SmartBookingPlatform. All services register with the Discovery Service for dynamic service discovery.

2.5 Architectural Observations

This architecture introduces several important characteristics:

- **Centralized entry point** – all external requests go through the API Gateway
- **Independent services** – each microservice encapsulates a business responsibility
- **Dynamic discovery** – service locations are resolved through Eureka
- **Loose coupling** – services communicate through contracts rather than direct dependencies
- **Scalability potential** – each service can be scaled independently in future versions

At the same time, this architecture also introduces distributed systems complexity, such as service coordination, fault handling, and observability concerns. These trade-offs are discussed later in the report.

3 Service Design

The SmartBookingPlatform is composed of several independent microservices, each responsible for a specific domain capability. This separation allows each service to evolve independently while maintaining a clear separation of concerns.

Each microservice follows a layered architecture composed of controllers, services, repositories, and domain models. This structure promotes maintainability and isolates business logic from infrastructure concerns.

The following sections describe the design and responsibilities of each service in the system.

3.1 Auth Service

The Auth Service is responsible for user authentication and identity management. It provides endpoints for user registration and login, generating JWT tokens that clients use to access protected resources.

The service performs the following responsibilities:

- User registration
- Password validation
- User authentication
- JWT token generation

When a user successfully authenticates, the service generates a signed JWT token containing user identity and role information. This token is later validated by the API Gateway for protected endpoints.

The Auth Service exposes the following main endpoints:

- `POST /api/auth/register`
- `POST /api/auth/login`

3.2 Catalog Service

The Catalog Service manages the marketplace catalog of offers and resources.

An **offer** represents a service provided by a provider, such as a hotel room listing or service appointment offering. Each offer can contain one or more **resources**.

A resource represents a specific bookable unit associated with an offer. For example, a hotel offer may contain multiple room resources, each representing an individual room that can be booked.

Resources include attributes such as:

- name
- price
- duration
- availability
- unavailable periods

The Catalog Service exposes endpoints for managing both offers and resources:

- `GET /api/offers`
- `GET /api/offers/{id}`
- `POST /api/offers`
- `DELETE /api/offers/{id}`
- `POST /api/resources`
- `GET /api/resources/{id}`
- `DELETE /api/resources/{id}`

3.3 Booking Service

The Booking Service is responsible for booking creation and validation.

Before creating a booking, the service verifies several conditions:

- The resource exists and is active
- The booking period does not overlap with an existing booking
- The requested time does not conflict with unavailable periods defined for the resource

To validate resource information, the Booking Service communicates with the Catalog Service using an OpenFeign client. This allows the service to retrieve resource metadata before confirming a booking.

The Booking Service exposes the following endpoints:

- `GET /api/bookings`
- `GET /api/bookings/{id}`
- `POST /api/bookings`
- `DELETE /api/bookings/{id}`

3.4 API Gateway

The API Gateway acts as the single entry point for all external client requests.

Instead of clients calling individual services directly, all requests are routed through the gateway. This allows the system to centralize several cross-cutting concerns, including:

- request routing
- authentication enforcement
- request filtering
- centralized logging

The gateway forwards requests to the appropriate services using route definitions. It also validates JWT tokens for protected endpoints before allowing requests to proceed.

3.5 Discovery Service

The Discovery Service provides service registration and discovery using Netflix Eureka.

When a microservice starts, it registers itself with the discovery server. Other services can then resolve its network location dynamically.

This approach removes the need for hardcoded service URLs and allows the system to support dynamic scaling and instance replacement.

4 Data Architecture and Service Isolation

In a microservices architecture, data ownership is a fundamental design principle. Each service is responsible for managing its own data, ensuring strong boundaries between services and preventing tight coupling at the database level.

4.1 Database per Service

The SmartBookingPlatform follows the **database-per-service** pattern. Each microservice owns its own PostgreSQL database instance:

- Auth Service → user accounts and authentication data
- Catalog Service → service offers and resources
- Booking Service → booking records and availability

This design ensures that services are fully independent and can evolve, scale, and deploy without impacting others.

Direct database access across services is strictly avoided. All interactions between services occur through well-defined APIs.

4.2 Database Schema Overview

The following diagram illustrates the database structure of each service and highlights the absence of cross-service database relationships.

```
erDiagram

%% ========================
%% AUTH SERVICE DATABASE
%% ========================
AUTH_USERS {
    UUID id PK
    string name
    string email
    string password
    string role
    boolean account_non_expired
    boolean account_non_locked
    boolean credentials_non_expired
    boolean enabled
```

```
}

%% ==========================
%% CATALOG SERVICE DATABASE
%% ==========================
SERVICE_OFFERS {
    UUID id PK
    string title
    string description
    string category
    string provider_name
    string location
    datetime created_at
}

SERVICE_RESOURCES {
    UUID id PK
    UUID offer_id FK
    string name
    decimal price
    int duration_in_minutes
    boolean active
}

UNAVAILABLE_PERIODS {
    UUID resource_id FK
    datetime start_time
    datetime end_time
}

SERVICE_OFFERS ||--o{ SERVICE_RESOURCES : has
SERVICE_RESOURCES ||--o{ UNAVAILABLE_PERIODS : has

%% ==========================
%% BOOKING SERVICE DATABASE
%% ==========================
BOOKINGS {
    UUID id PK
    UUID resource_id
    string customer_name
    string customer_email
    datetime start_time
    datetime end_time
    string status
    datetime created_at
}
```

Each microservice owns its own independent schema, with no shared tables across services.

Relationships exist only within service boundaries. For example, the Catalog Service maintains foreign key relationships between `ServiceOffer` and `ServiceResource`.

However, cross-service relationships are not enforced at the database level. The Booking Service stores only a reference (`resourceId`) to resources managed by the Catalog Service, without any foreign key constraint.

This design enforces strict service isolation and ensures that services communicate exclusively through APIs rather than direct database access.

This approach avoids distributed transactions and tight coupling at the database level, but shifts the responsibility of maintaining consistency to the application layer.

4.3 Rationale

This approach was chosen for the following reasons:

- **Loose coupling** — services are not tied to a shared schema
- **Independent evolution** — each service can change its data model without breaking others
- **Fault isolation** — failures in one database do not directly affect others

While a shared database would simplify queries across domains, it would introduce tight coupling and make the system harder to maintain and scale.

4.4 Data Consistency Across Services

Because each service owns its data, cross-service consistency becomes a challenge.

For example:

- The Booking Service depends on resource data from the Catalog Service
- However, it does not have direct access to the Catalog database

To ensure correctness, the system currently uses synchronous validation:

- Booking Service calls Catalog Service before creating a booking
- Resource availability and constraints are validated in real time

This guarantees strong consistency at the cost of increased coupling and reduced availability.

4.5 Trade-offs

Aspect	Impact
Strong consistency	Ensures bookings are always validated against the latest data
Service coupling	Booking depends on Catalog availability
Operational complexity	Requires careful API design instead of direct queries

4.6 Schema Management

Each service manages its own schema independently using JPA/Hibernate.

In the current version, all services use `ddl-auto=create-drop`:

```
spring.jpa.hibernate.ddl-auto=create-drop
```

This means the schema is created on startup and **dropped entirely on shutdown**, destroying all data. This was a deliberate choice during development to avoid stale test data accumulating across Postman sessions.

This setting is only acceptable in development. In production it would cause catastrophic data loss on every restart. Before any production deployment, this must be replaced with a migration tool such as Flyway, which provides controlled, versioned, and reversible schema changes.

The choice between Flyway and Liquibase is straightforward for a Spring Boot project at this stage: Flyway is simpler, SQL-based, and has excellent Spring Boot auto-configuration. Liquibase offers more power (XML/YAML formats, rollback support) but adds unnecessary complexity at this scale.

4.7 Future Evolution: Event-Driven Data Consistency

The current synchronous approach introduces a hard dependency between services.

To reduce this coupling, a future improvement involves adopting an event-driven model using Apache Kafka.

In this model:

- Catalog Service publishes events (e.g., `ResourceCreated`, `ResourceUpdated`)
- Booking Service consumes these events and maintains a local copy of resource data

This shifts consistency from **strong consistency** to **eventual consistency**, reducing runtime dependencies between services.

However, not all data can be safely replicated:

- **Stable data** (name, price, duration) — safe to replicate
- **Dynamic data** (availability, unavailable periods) — requires real-time validation

A hybrid approach is therefore recommended: replicate stable data asynchronously while keeping synchronous validation for time-sensitive constraints.

4.8 Lessons Learned

- Data ownership is the foundation of microservices — without it, services are not truly independent
- Strong consistency simplifies logic but introduces tight coupling
- Event-driven architectures reduce coupling but introduce new challenges (staleness, complexity)
- Not all data should be treated equally — distinguishing between stable and dynamic data is critical

5 Inter-Service Communication and Resilience

In a microservices architecture, services must communicate with each other to fulfill business operations. This communication introduces network latency, partial failures, and service dependency risks.

The SmartBookingPlatform addresses these challenges using synchronous communication via OpenFeign combined with resilience mechanisms provided by Resilience4j. Synchronous communication was chosen for simplicity at this stage, with asynchronous event-driven patterns considered for future iterations.

5.1 Service Communication

Inter-service communication is performed using REST APIs through OpenFeign clients.

The Booking Service depends on the Catalog Service to retrieve resource information before creating a booking. This interaction is implemented using a Feign client abstraction, which simplifies HTTP communication and integrates natively with Spring Cloud.

CatalogClient interface

```
@FeignClient(name = "catalog-service")
public interface CatalogClient {

    @GetMapping("/api/resources/{id}")
    ServiceResourceResponseDTO getResourceById(@PathVariable UUID id);
}
```

This abstraction allows the Booking Service to call the Catalog Service as if it were a local method, while the underlying communication remains HTTP-based.

5.2 Failure Scenarios

Because services depend on each other, several failure scenarios must be considered:

- Catalog Service is unavailable
- Network latency or timeout
- Partial failures (intermittent errors)

Without proper handling, these failures could propagate across services and degrade the reliability of the entire system.

5.3 Resilience Mechanisms

To mitigate these risks, the system uses Resilience4j to implement three mechanisms: Retry, Circuit Breaker, and Time Limiter. These are applied specifically to `fetchResource()`, the method responsible for calling the Catalog Service.

```
@CircuitBreaker(name = "catalogService", fallbackMethod = "catalogFallback")
@Retry(name = "catalogService")
public ServiceResourceResponseDTO fetchResource(UUID resourceId) {
    return catalogClient.getResourceById(resourceId);
}
```

5.3.1 Retry

Retry automatically re-executes failed requests a limited number of times before propagating the error. Useful for transient failures such as temporary network issues.

```
resilience4j:
  retry:
    instances:
      catalogService:
        maxAttempts: 3
        waitDuration: 500ms
```

5.3.2 Circuit Breaker

The circuit breaker prevents repeated calls to a failing service by opening the circuit after a failure threshold is reached. When open, calls to the Catalog Service are temporarily blocked, reducing load and preventing cascading failures.

```
resilience4j:
  circuitbreaker:
    instances:
      catalogService:
        slidingWindowSize: 5
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
```

These values were chosen for demonstration purposes and fast feedback during development.

A sliding window of 5 requests allows the circuit breaker to react quickly to failures, but is too small for production scenarios, where larger windows provide more stable failure rate calculations.

The 50% failure threshold ensures that the circuit opens only when a significant portion of requests fail, while the 10-second open state prevents immediate retry storms against a failing service.

5.3.3 Time Limiter

The time limiter ensures that requests do not hang indefinitely. If a response is not received within the configured duration, the request is aborted.

```
resilience4j:
  timelimiter:
    instances:
      catalogService:
        timeoutDuration: 2s
```

Important Note The time limiter (2s) applies to each individual request attempt, not to the entire retry sequence.

This means that multiple retries can still occur, as long as each individual request completes within the timeout window. As a result, the total execution time may exceed the configured timeout when retries are involved.

5.4 Fallback Strategy

When `fetchResource()` fails after all retries, or the circuit breaker is open, `catalogFallback()` is triggered and throws a `ResourceUnavailableException`. The booking request is rejected with a clear error message.

```
public ServiceResourceResponseDTO catalogFallback(UUID resourceId, Throwable ex) {
    throw new ResourceUnavailableException("Catalog service unavailable");
}
```

The execution order is:

1. `fetchResource()` is called
2. Resilience4j intercepts — retries up to 3 times
3. If still failing — `catalogFallback()` fires and throws `ResourceUnavailableException`
4. Booking request is rejected with a clear error message

5.5 Design Decisions

Decision	Rationale
Synchronous communication via OpenFeign	Simpler to implement and debug at this stage; sufficient for current load
Resilience4j only where inter-service dependency exists	Avoids unnecessary complexity in services that do not call other services
Reject booking when Catalog is unavailable	Prevents bookings with missing or unvalidated resource data
Circuit breaker to block calls on failure	Reduces system load and prevents cascading failures during outages

5.6 Trade-offs

Trade-off	Impact
Increased configuration overhead	Each resilience mechanism requires tuning per service instance
Potential latency due to retries	3 attempts with 500ms between each adds up to 1s of wait time, plus the duration of each individual request. The time limiter (2s) applies per call, not to the whole retry sequence, so all 3 attempts can execute as long as each individual request resolves within 2s
Synchronous coupling	If the Catalog Service is unavailable, booking creation is completely blocked. From a user perspective, this results in immediate request failure, typically surfaced as an HTTP error response. While this ensures data consistency (no booking without validation), it also reduces system availability and resilience.

5.7 Future Improvements

5.7.1 Kafka and Event-Driven Communication

Apache Kafka will be introduced in V2 to reduce synchronous coupling between services. The plan is to use event-driven data replication: the Catalog Service publishes events such as `ResourceCreated`, `ResourceUpdated`, and `ResourceDeactivated`, which the Booking Service consumes to maintain its own local copy of resource data. This removes the need to call the Catalog Service synchronously on every booking request.

However, this approach does not fully eliminate the dependency problem. Availability state and unavailable periods change frequently, and a local copy may be stale by the time a booking is created. Two categories of data need to be treated differently:

- **Stable data** (name, price, duration) — safe to replicate via Kafka; staleness is acceptable
- **Availability state and unavailable periods** — changes frequently; a stale copy risks accepting bookings that should be rejected

A pragmatic V2 approach would be a hybrid model: replicate stable resource data via Kafka into the Booking Service, and keep a lightweight synchronous call only for real-time availability checks.

This highlights a key trade-off in distributed systems: reducing coupling through asynchronous communication often introduces data consistency challenges.

5.7.2 Other Improvements

- Revisit the current use of RabbitMQ (used for logging) and assess whether to consolidate with Kafka or keep it separate
- Improve fallback responses to return partial booking data where safe to do so

6 Security Architecture

SmartBookingPlatform uses **stateless JWT-based authentication**, with all security concerns centralized at the API Gateway.

All client interactions go through the API Gateway, including authentication requests. During login, the gateway forwards credentials to the Auth Service, which performs validation and issues a signed JWT token.

No server-side sessions are maintained; every request is validated independently using a signed token.

This design keeps internal microservices isolated from clients and avoids duplicating security logic across services.

> **Current implementation:** The JWT secret is stored in a local `.env` file and shared between the Auth Service and API Gateway.
>
> **Future improvement:** This approach risks configuration drift as the system grows. Centralizing secret management (e.g. HashiCorp Vault) should be a V2 priority.

6.1 Authentication Flow

1. Client sends credentials to `POST /api/auth/login` via the API Gateway
2. Gateway forwards the request to the Auth Service
3. Auth Service validates the credentials and generates a signed JWT
4. Token is returned to the client through the gateway
5. Client attaches the token to all subsequent requests:

   ```
   Authorization: Bearer <JWT_TOKEN>
   ```

6.2 Example Requests

The following examples illustrate how authentication is performed against the API Gateway.

User Registration

```
POST /api/auth/register
Content-Type: application/json

{
  "name": "Joana Mendes",
  "email": "joana.mendes@example.com",
  "password": "securePassword123",
  "role": "PROVIDER"
}
```

Response `201 OK`

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI4ZDc0ZTRjZ...",
  "issuedAt": "2026-03-19T16:28:45Z",
  "expiresAt": "2026-03-19T17:28:45Z",
  "tokenType": "Bearer"
}
```

User Login

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "joana.mendes@example.com",
  "password": "securePassword123"
}
```

Response `200 OK`

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI4ZDc0ZTRjZ...",
  "issuedAt": "2026-03-19T16:29:03Z",
  "expiresAt": "2026-03-19T17:29:03Z",
  "tokenType": "Bearer"
}
```


6.3 JWT Token Structure

Each token is digitally signed to guarantee integrity and prevent tampering. It carries the minimum information needed for validation and access control:

Claim	Purpose
User ID	Identifies the requester
Email	User identity
Role	Drives authorization decisions
Expiration	Limits token lifetime

JWT tokens in this system have a limited lifetime (1 hour), after which clients must authenticate again to obtain a new token.

6.4 Authorization at the Gateway

For every protected request, the gateway follows this sequence:

1. Extracts the token from the `Authorization` header
2. Validates the signature and expiration
3. If valid → forwards the request to the target service
4. If invalid → rejects with the appropriate HTTP error status

This ensures that internal services only receive pre-validated requests and do not need to implement full authentication logic.

6.5 JWT Validation Implementation

JWT validation is implemented at the API Gateway level using Spring Security within a reactive (WebFlux) context, consistent with Spring Cloud Gateway's non-blocking architecture. A custom security filter intercepts incoming requests, extracts the JWT token from the `Authorization` header, and validates its signature and expiration using the shared secret. This filter is implemented as a reactive `WebFilter`, the WebFlux equivalent of servlet-based filters:

```java
@Component
public class JwtAuthenticationFilter implements WebFilter {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, WebFilterChain chain)
        { ... }
}
```

The filter is registered at the authentication stage of the security chain:

```
.addFilterAt(jwtAuthenticationFilter, SecurityWebFiltersOrder.AUTHENTICATION)
```

Unlike the Auth Service, the gateway does not query the database to load user details. It assumes that a valid token is sufficient and stores a minimal authentication object in the reactive security context using `ReactiveSecurityContextHolder`.

If the token is valid, the request proceeds through the routing chain. If the token is missing or invalid, the filter continues the chain without setting authentication, and Spring Security's authorization layer rejects the request before it reaches any internal service.

Session management is stateless by design. CSRF, form login, and HTTP basic authentication are all explicitly disabled.

Stateless behavior is inherent to the reactive model — no server-side session is created.

Authentication is established per request via the JWT filter and stored only in the reactive `SecurityContext` for the duration of that request.

6.6 Token Propagation

After validation, the API Gateway forwards the original JWT token to downstream services. Internal services do not re-validate the token but may extract information (such as user ID or role) if needed for business logic. This approach assumes trust within the internal network and avoids duplicating security logic across services. Note that this is a deliberate simplification acceptable for the current single-host deployment — in a zero-trust environment this assumption would need to be revisited.

6.7 Role-Based Authorization

The JWT token contains a role claim with three possible values: `CLIENT`, `PROVIDER`, and `ADMIN`. Roles are assigned at registration and embedded in the token, but no authorization logic has been implemented yet. All authenticated requests are treated equally regardless of role. This is a known gap. A future implementation should enforce, at minimum:

- Only `PROVIDER` can create or delete offers and resources
- Only `CLIENT` can create bookings
- Only `ADMIN` can manage users

6.8 Public vs Protected Endpoints

Endpoint	Access
`POST /api/auth/register`	Public
`POST /api/auth/login`	Public
All other routes	Requires valid JWT

6.9 Key Design Decisions

These are the decisions worth remembering for future projects:

The main idea to carry forward: **push security to the edge and minimize security logic inside services.**

Decision	Rationale
Centralize security at the gateway	Single enforcement point — no duplication across services
Stateless JWT over server sessions	Enables independent scaling; no shared session store needed
Delegate credential validation to Auth Service	Keeps the gateway focused while isolating authentication logic
Never expose internal services directly	Clients always go through the gateway, reducing attack surface

6.10 Lessons Learned

- Define and enforce role boundaries before the system grows — retrofitting authorization logic across multiple services is significantly harder than building it in from the start

- The gateway is the ideal place to enforce coarse-grained authorization fine-grained rules (e.g. a provider can only delete their own offers) belong inside the service

- Reactive security (WebFlux) differs fundamentally from the traditional servlet-based model used in Spring MVC. The key difference lies in how requests are processed.

  In Spring MVC, each request is handled by a dedicated thread. Security relies on the Servlet API, where filters extend `OncePerRequestFilter`, the chain is defined by `SecurityFilterChain`, and authentication is stored in `SecurityContextHolder` using thread-local storage.

  In contrast, WebFlux is fully asynchronous and non-blocking. A single request may be processed across multiple threads, which makes thread-local storage unreliable. Because of this, the entire security model changes.

  Instead of servlet filters, WebFlux uses `WebFilter`.

  The security chain is defined by `SecurityWebFilterChain`, and authentication is stored in `ReactiveSecurityContextHolder`, which propagates through the reactive pipeline (`Mono`/`Flux`) rather than being bound to a single thread.

  These two models are not interchangeable. A WebFlux application cannot rely on servlet-based security mechanisms, and attempting to mix both approaches will lead to incorrect or non-functional behavior.

7 Testing Strategy

7.1 Overview

Testing in the SmartBookingPlatform is organised into two categories: unit tests per service and load tests using k6.

Unit tests validate business logic and HTTP behaviour in isolation, without starting the full application context where possible. Load tests simulate concurrent users interacting with the system through the API Gateway to evaluate performance and stability under load.

Integration tests — tests that validate end-to-end flows across multiple services with real databases and network calls — were not implemented in V1. This is a known gap, discussed in Section 7.5.

7.2 Testing Layers

All unit tests are organised around distinct testing layers. Each layer has a specific scope, a set of tools, and a clear reason for existing independently of the others.

Layer	What it tests	Tools	Requires infra
HTTP	Request mapping, status codes, JSON responses. The service layer is mocked — no business logic executes.	`@WebMvcTest`, MockMvc	No
Business Logic	Service rules, validation, exception throwing. The repository and external clients are mocked — no database or network.	Mockito, JUnit 5	No
Exception Mapping	How exceptions thrown by the service are translated into HTTP responses by the GlobalExceptionHandler.	`standaloneSetup()`, MockMvc	No
Validation	Bean Validation constraints on DTOs, tested directly without Spring context.	Jakarta Validator	No
Infrastructure	Spring context startup and RabbitMQ publish smoke test. No CRUD, no data modification.	`@SpringBootTest`	Yes

The key principle: each layer mocks everything below it. The HTTP layer does not know about the database. The business logic layer does not make real network calls. This keeps tests fast, focused, and independent of infrastructure.

7.3 Unit Tests

Each service follows the same layered structure. All tests use JUnit 5 and Mockito.

7.3.1 Auth Service

The Auth Service has 18 tests across 4 classes, all passing.

Class	Tests	Layer	Focus
AuthControllerTest	5	HTTP	Endpoint mapping and responses
AuthServiceImplTest	5	Business Logic	Registration and authentication rules
JwtServiceTest	6	Business Logic	Token generation and validation
LogEventProducerTest	1	Infrastructure	RabbitMQ publish smoke test
Total	**18**		

AuthControllerTest uses `@WebMvcTest` to test the HTTP layer in isolation. Spring Security is disabled and the service is mocked via `@TestConfiguration`. `JwtAuthenticationFilter` is explicitly excluded because it depends on beans not loaded in the `@WebMvcTest` context. Scenarios covered:

- Successful registration returns HTTP 201 with token in response body
- Successful login returns HTTP 200 with token in response body
- Duplicate email returns HTTP 409
- Invalid request payload returns HTTP 400 (Bean Validation)
- Invalid credentials return HTTP 401

AuthServiceImplTest tests business logic in pure isolation — no Spring context, no database, no real JWT signing. Scenarios covered:

- Successful registration encodes password, saves user, generates token
- Duplicate email throws `EmailAlreadyExistsException` and never calls `repository.save()`
- Successful authentication returns token with correct user data
- Unknown email throws `UsernameNotFoundException`
- Wrong password throws `InvalidCredentialsException`

JwtServiceTest validates JWT logic without Spring context. Scenarios covered:

- Generated token is valid and claims are extracted correctly
- Invalid token returns false without throwing
- Token signed with wrong secret is rejected
- Expired token (100ms lifetime) is rejected after expiry
- Tampered token (last character modified) is rejected

The secret mismatch and tamper tests are particularly valuable — they verify that the JWT signature mechanism actually protects against forgery, not just that the happy path works.

LogEventProducerTest uses `@SpringBootTest` to verify that the RabbitMQ producer does not throw on publish. The test passes because `assertDoesNotThrow` only validates that no exception propagates — actual delivery is not verified.

7.3.2 Catalog Service

The Catalog Service has 32 tests across 7 classes, all passing.

Class	Tests	Layer	Focus
CatalogServiceApplicationTests	1	Infrastructure	Context loads (requires DB)
ServiceOfferControllerTest	7	HTTP	Offer endpoints
ServiceResourceControllerTest	4	HTTP	Resource endpoints
GlobalExceptionHandlerStandaloneTest	5	Exception Mapping	Error responses
ServiceOfferServiceImplTest	6	Business Logic	Offer rules
ServiceResourceServiceImplTest	7	Business Logic	Resource rules
LogEventProducerTest	1	Infrastructure	RabbitMQ smoke test
Total	**32**		

Infrastructure-dependent tests: `CatalogServiceApplicationTests` and `LogEventProducerTest` require active infrastructure — PostgreSQL and RabbitMQ respectively. Neither performs real CRUD operations or modifies data. Both tests fail when infrastructure is unavailable. This was a conscious decision — running these tests requires the full platform to be active via Docker, which is the normal development setup. The trade-off is that these tests cannot run in isolation without infrastructure, which would need to be addressed before introducing a CI/CD pipeline in V2.

ServiceOfferControllerTest uses `@WebMvcTest` with the service mocked via `@TestConfiguration`. Scenarios covered:

- `GET /api/offers` returns all offers
- `GET /api/offers` with `includeResources=true` returns offers with nested resources
- `GET /api/offers/{id}` returns HTTP 200 when found, HTTP 404 when not found
- `POST /api/offers` returns HTTP 201 with Location header
- `DELETE /api/offers/{id}` returns HTTP 204 when deleted, HTTP 404 when not found

ServiceResourceControllerTest follows the same pattern. Scenarios covered:

- `POST /api/resources` returns HTTP 201 with Location header
- `GET /api/resources/{id}` returns HTTP 200 when found, HTTP 404 when not found
- `DELETE /api/resources/{id}` returns HTTP 204

GlobalExceptionHandlerStandaloneTest uses `standaloneSetup()` instead of `@WebMvcTest`, making it faster and fully independent of the Spring context. Scenarios covered:

- Bean Validation failure → HTTP 400 with structured error body
- Database integrity violation → HTTP 400
- Illegal argument → HTTP 400
- Malformed JSON → HTTP 400

- Unhandled exception → HTTP 500

ServiceOfferServiceImplTest tests offer business logic with pure Mockito. Scenarios covered:

- `getAll()` returns mapped list from repository
- `getById()` returns offer with resources when found, never calls resource repository when offer not found
- `create()` saves offer and returns mapped response
- `delete()` calls `deleteById()` when offer exists, returns false and never calls delete when not found

ServiceResourceServiceImplTest tests resource business logic. Scenarios covered:

- Successful creation validates offer exists before saving
- Throws exception when offer not found, never calls `repository.save()`
- Throws exception when unavailable period has end before start
- `getByOffer()`, `getById()`, and `delete()` delegate correctly to repository

7.3.3 Booking Service

The Booking Service has 22 tests across 6 classes, all passing.

Class	Tests	Layer	Focus
BookingServiceApplicationTests	1	Infrastructure	Context loads (requires DB)
BookingControllerTest	6	HTTP	Booking endpoints
GlobalExceptionHandlerStandaloneTest	5	Exception Mapping	Error responses
BookingServiceImplTest	6	Business Logic	Booking rules and conflict detection
EndAfterStartValidationTest	3	Validation	DTO time constraint
LogEventProducerTest	1	Infrastructure	RabbitMQ smoke test
Total	**22**		

BookingControllerTest uses `@WebMvcTest` with the service mocked via `@TestConfiguration`. Scenarios covered:

- `GET /api/bookings` returns all bookings
- `GET /api/bookings/{id}` returns HTTP 200 when found, HTTP 404 when not found
- `POST /api/bookings` returns HTTP 201 on success
- `DELETE /api/bookings/{id}` returns HTTP 204 when deleted, HTTP 404 when not found

GlobalExceptionHandlerStandaloneTest uses `standaloneSetup()` and covers booking-specific exceptions not present in other services:

- Bean Validation failure → HTTP 400
- Database integrity violation → HTTP 400
- `BookingConflictException` → HTTP 409
- `ResourceUnavailableException` → HTTP 503
- Unhandled exception → HTTP 500

The HTTP 503 for resource unavailability is specific to the Booking Service — it signals that a dependency (Catalog Service) is down, not a client error.

BookingServiceImplTest tests booking business logic with pure Mockito. `CatalogClient` is mocked, allowing validation logic to be tested without a running Catalog Service. Scenarios covered:

- Successful booking creation when no conflicts exist
- Throws `BookingConflictException` when time slot overlaps an existing booking — `repository.save()` is never called
- Throws `ResourceUnavailableException` when resource is inactive
- Throws `ResourceUnavailableException` when booking time conflicts with an unavailable period defined on the resource
- Successful deletion returns true and calls `deleteById()`
- Deletion of non-existing booking returns false and never calls `deleteById()`

EndAfterStartValidationTest validates the custom `@EndAfterStart` annotation directly using the Jakarta Bean Validator without Spring context. Scenarios covered:

- Valid booking passes when end time is after start time
- Validation fails when end time is before start time
- Validation fails when end time equals start time

This is the only test class that uses the Jakarta Validator directly — a clean approach for testing custom constraint annotations in isolation.

7.4 Test Summary

Service	Classes	Tests	Result
Auth Service	4	18	18/18 passing
Catalog Service	7	32	32/32 passing
Booking Service	6	22	22/22 passing
Total	**17**	**72**	**72/72 passing**

7.5 Test Gaps and Limitations

- **No integration tests** — all tests mock dependencies. There is no test that validates the full booking flow across Auth, Catalog, and Booking services with real databases and real HTTP calls between services.
- **No gateway tests** — JWT validation at the gateway level is tested manually via Postman but not automated.
- **No RabbitMQ delivery validation** — log events are published but delivery to the broker is not asserted in tests.
- **No contract tests** — the DTO contract between Booking and Catalog services is not formally tested. If the resource response DTO changes in Catalog, the Booking Service will fail at runtime, not at test time. Consumer-driven contract tests (e.g. Pact) should be considered for V2.
- **Infrastructure tests require Docker** — tests using `@SpringBootTest` fail without active PostgreSQL and RabbitMQ. This must be addressed before introducing CI/CD in V2.

7.6 Load Testing with k6

Load tests were written using k6 and executed against the full running platform through the API Gateway. Three test scenarios were defined, each targeting a different part of the system.

7.6.1 Auth Baseline Test

Goal: Measure authentication performance under concurrent load.

Configuration: 10 virtual users, 30 seconds.

Strategy: Each VU registers once using a unique email, then repeatedly performs login requests for the duration of the test. This isolates login latency as the primary metric.

Results:

Metric	Value
Total requests	232
HTTP failures	0%
Avg response time	348ms
Median response time	228ms
p(90)	302ms
p(95)	404ms
Max response time	2.8s
Check passed (login < 300ms)	92%

Observations: All requests reached the server successfully with zero HTTP failures. 92% of login responses were within the 300ms threshold. The maximum of 2.8s is likely a JVM warm-up spike or garbage collection pause on first execution — subsequent requests settled significantly lower. The p(95) of 404ms suggests the system handles sustained load well but has occasional outliers under concurrent pressure.

7.6.2 Catalog Load Test

Goal: Measure catalog retrieval performance under higher concurrency.

Configuration: 20 virtual users, 30 seconds.

Strategy: Each VU authenticates once and repeatedly calls the offers endpoint through the API Gateway.

Results:

Metric	Value
Total requests	912
HTTP failures	0%
Avg response time	180ms
Median response time	180ms
p(90)	316ms
p(95)	747ms
Max response time	1.48s
Check passed (response < 500ms)	95%

Observations: The Catalog Service handled 20 concurrent users with zero HTTP failures. Average and median response times are close (180ms each), indicating stable behaviour under normal load. However, the p(95) of 747ms reveals that under peak concurrency, some requests degraded significantly. This gap between median and p(95) suggests the system begins to show pressure at higher concurrency — likely at the database query level, since catalog retrieval involves no cross-service calls.

7.6.3 Booking Flow Test

Goal: Simulate a realistic provider workflow under concurrent load and validate booking conflict detection.

Configuration: 5 virtual users, 30 seconds.

Strategy: Each VU registers, authenticates, creates an offer, creates a resource, and repeatedly attempts to create bookings. Time slots are offset per VU to reduce intentional conflicts, though overlap can still occur across iterations.

Results:

Metric	Value
Total iterations	116
HTTP failures	45.34%
Check passed	79%
Avg response time	150ms
p(90)	221ms
p(95)	303ms

Observations: The 45% HTTP failure rate does not indicate system instability. The majority of these failures are HTTP 409 responses — the system correctly rejecting duplicate booking attempts for the same time slot across iterations.

This result exposed an important monitoring insight: aggregating all non-2xx responses as failures produces misleading metrics. In a production monitoring setup, HTTP status codes must be segmented by category:

- **5xx** — system errors requiring immediate attention
- **4xx (409, 404)** — expected business rejections, monitored separately as business metrics
- **4xx (401, 403)** — security violations, monitored as security events

Treating 409 as a failure in a dashboard would trigger false alarms in a real booking system where conflicts are an expected and frequent outcome.

7.7 Lessons Learned

- Mock dependencies at the right layer — `@WebMvcTest` for HTTP behaviour, pure Mockito for business logic. Mixing the two leads to slow, fragile tests
- When the code evolves, tests must follow — both failures found in the Auth Service (wrong status code, wrong exception type) were bugs in the tests, not the code. Tests that are not maintained become noise
- Contract between services is a testing gap — if a DTO changes in one service, only a runtime failure reveals the break. Consumer-driven contract tests (e.g. Pact) solve this and should be considered for V2
- HTTP status code semantics matter in monitoring — a 409 is not a failure, it is a business rule enforced correctly. Monitoring dashboards must distinguish between system errors and expected business responses
- Integration tests are the missing layer — unit tests verify components in isolation but give no confidence that services work correctly together. A basic end-to-end test covering the full booking flow would significantly increase confidence
- Custom constraint annotations should be tested directly using the Jakarta Validator without Spring context — faster, simpler, and more focused than loading the full framework

8 Architectural Trade-offs

8.1 Microservices vs Monolith (Solo Developer Perspective)

The decision to adopt a microservices architecture for this project was deliberate, ut not the most pragmatic choice for a solo developer. The goal was not only to build a working system, but to gain hands-on experience with distributed system patterns used in real-world architectures.

Comparison

Aspect	Monolith / Modular Monolith	Microservices (current)
Development speed	Fast — single codebase	Slow — multiple services to configure
Debugging	Simple — local method calls	Complex — network calls between services
Data consistency	Native FK constraints, single transaction	No FK across services, manual validation
Infrastructure	Minimal	Eureka, API Gateway, resilience patterns
Deployment	Single deployable unit	Independent per service
Team scalability	Limited	High — teams work independently
Learning value	Lower for distributed patterns	High — forces real trade-off decisions

Reflection

For a real-world project developed by a single engineer, a modular monolith would likely be the preferred approach — organising code into clearly bounded modules (auth, catalog, booking) within a single deployable application. This preserves separation of concerns and makes future extraction into microservices straightforward, without paying the operational cost of distributed systems upfront.

Microservices become a justified choice when three factors align:

- **Team size** — independent teams working on separate domains
- **Deployment independence** — services need to be released separately
- **Scalability** — specific components require independent scaling

This project highlights that architecture is context-dependent — the best solution is not the most complex one, but the one that fits the problem and the team.

8.2 Synchronous vs Asynchronous Communication

The SmartBookingPlatform currently uses synchronous communication between services, implemented via REST APIs using OpenFeign. When creating a booking, the Booking Service calls the Catalog Service directly and waits for a response before proceeding.

Current Approach: Synchronous

Synchronous communication was chosen for simplicity and real-time validation. Booking creation requires up-to-date availability data — eventual consistency would risk double-booking.

To mitigate the coupling introduced by this approach, resilience patterns were applied using Resilience4j (covered in detail in Chapter 5): Retry for transient failures, Circuit Breaker to prevent cascading failures, and Time Limiter to avoid indefinitely blocked requests. These improve stability but do not remove the fundamental dependency — if the Catalog Service is unavailable, booking creation is still blocked.

Comparison

Aspect	Synchronous (current)	Asynchronous (Kafka)
Consistency	Strong — always up to date	Eventual — data may lag
Coupling	Tight — services must be available	Loose — services are independent
Debugging	Simple — linear request flow	Complex — non-linear event flow
Resilience	Lower — failures propagate	Higher — failures are isolated
Complexity	Low	High (ordering, idempotency)

Reflection

Not all data requires the same approach. A hybrid model is planned for V2:

- **Asynchronous (Kafka)** — replicate stable data (resource name, price, duration) where staleness is acceptable
- **Synchronous (OpenFeign)** — keep real-time calls only for availability validation, where a stale answer risks double-booking

The key lesson: communication patterns should be chosen based on the nature of the data, not applied uniformly across the system.

8.3 Strong vs Eventual Consistency

In a distributed system, maintaining data consistency across services is a fundamental challenge. Each service owns its own data, which removes the possibility of enforcing consistency through a single transactional boundary.

In the SmartBookingPlatform, this problem is most visible in the booking flow, where the Booking Service depends on up-to-date resource availability data from the Catalog Service.

Current Approach: Strong Consistency

The system enforces strong consistency through synchronous validation, as described in Section 8.2. This was a deliberate choice driven by the nature of availability data rather than by the communication model itself.

This guarantees that bookings are always validated against the most recent state, preventing conflicting reservations and preserving core business invariants.

Comparison

Aspect	Strong Consistency (current)	Eventual Consistency (Kafka)
Data accuracy	Always up to date	May be temporarily stale
User impact	No invalid bookings	Risk of conflicts (e.g. double-booking)
Coupling	High — depends on Catalog Service	Low — services operate independently
Performance	Slower — requires real-time calls	Faster — no blocking requests
Failure handling	Fails fast if dependency is down	More resilient — continues operating

Reflection

Strong consistency was the correct choice for booking creation.

Availability data is highly dynamic, and even small delays in propagation can lead to incorrect system behavior. Accepting stale data in this context would introduce the risk of double-booking, which directly violates the core business constraint of the system.

However, enforcing strong consistency introduces tight coupling.

The Booking Service becomes dependent on the availability and responsiveness of the Catalog Service, meaning that failures in one service directly affect the ability to process bookings.

A full transition to eventual consistency is not appropriate for all data.

A more effective approach is to distinguish between different types of data:

- **Dynamic data (availability, unavailable periods)** — requires strong consistency and real-time validation
- **Stable data (resource name, price, duration)** — can tolerate staleness and be replicated asynchronously

In a future version, this leads to a hybrid model:

- Use Kafka to replicate stable data into the Booking Service
- Keep synchronous validation only for time-sensitive constraints

This reduces runtime coupling while preserving correctness where it matters most.

The key lesson: consistency requirements should be driven by business rules, not by architectural preference — not all data needs to be strongly consistent, but critical invariants must always be protected.

8.4 Centralized Gateway vs Distributed Security

Security in the SmartBookingPlatform is centralized at the API Gateway. All incoming requests pass through the gateway, where JWT tokens are validated before being routed to internal services.

Internal services do not implement their own authentication mechanisms and assume that all incoming requests have already been validated.

Comparison

Aspect	Centralized Security (current)	Distributed Security
Implementation	Single point (API Gateway)	Each service implements security
Consistency	Uniform across all services	Risk of inconsistent policies
Complexity	Lower — logic defined once	Higher — duplicated across services
Performance	Efficient — validated once per request	Re-validation in each service
Security model	Trust internal network	Zero-trust between services
Flexibility	Limited per-service customization	Fine-grained control per service

Reflection

Centralizing security at the API Gateway simplifies the system significantly.

Authentication logic is implemented once and enforced consistently across all services. This reduces duplication and avoids the risk of configuration drift between services.

This approach also aligns well with the current architecture, where all external traffic is routed through a single entry point.

However, this design introduces an implicit trust model.

Internal services assume that any request received from the gateway is valid and do not perform additional verification. In practice, this means that if an attacker gains access to the internal network — bypassing the gateway — they could invoke any service without authentication.

This assumption is acceptable in the current single-host development setup, but becomes a risk in distributed environments (e.g., cloud or containerized deployments) where services share a network boundary.

Another limitation is the absence of role-based authorization.

Although roles are included in the JWT token, all authenticated requests are treated equally. Authorization is therefore limited to a coarse-grained distinction between authenticated and unauthenticated access.

Future direction

- Introduce role-based authorization at the gateway level (e.g., restrict certain endpoints to `PROVIDER` or `ADMIN`)
- Consider propagating user context more explicitly to services when fine-grained authorization is required
- Evaluate a zero-trust approach where services validate tokens independently in more complex deployments

The key lesson: centralizing security reduces complexity and ensures consistency, but introduces implicit trust boundaries. As systems scale, these boundaries must be revisited and hardened.

8.5 Database per Service vs Shared Database

The SmartBookingPlatform follows the database-per-service pattern, where each microservice owns its own database and schema.

This means that:

- The Auth Service manages user data
- The Catalog Service manages offers and resources
- The Booking Service manages bookings

No service is allowed to directly access another service's database. All interactions must occur through APIs.

Comparison

Aspect	Database per Service (current)	Shared Database
Coupling	Loose — services are independent	Tight — shared schema across services
Data ownership	Clear boundaries per service	Blurred ownership
Schema evolution	Independent per service	Changes affect multiple services
Consistency	Managed at application level	Enforced via database constraints (FK, transactions)
Development speed	Slower — requires APIs for data access	Faster — direct queries across domains
Scalability	High — services scale independently	Limited — database becomes bottleneck

Reflection

The database-per-service pattern enforces strong service boundaries.

Each service owns its data and is responsible for its integrity, which prevents tight coupling at the database level and allows services to evolve independently.

However, this design shifts the responsibility for data consistency from the database to the application layer.

In this system, the Booking Service stores only a reference (`resourceId`) to resources managed by the Catalog Service, without any foreign key constraint. As a result, correctness depends on explicit validation through service calls.

This introduces additional complexity:

- Cross-service validation must be implemented manually
- Failures in dependent services impact data correctness
- There is no single transactional boundary across services

In contrast, a shared database would allow:

- Foreign key constraints across domains
- Strong consistency enforced at the database level
- Simpler queries and joins across entities

However, this would tightly couple services to a shared schema, making independent evolution and deployment significantly harder.

For this project, database-per-service was the correct architectural choice.

Although it introduces additional complexity, it aligns with the goal of building a true microservices architecture and exposes the real challenges of distributed data management.

For a simpler system or a solo-developed application, a shared database within a modular monolith would likely be more pragmatic — as discussed in Section 8.1.

9 Observability and Logging

Observability in a distributed system means being able to answer the question: *"What is the system doing right now, and why did it behave that way?"*

In V1, observability is addressed through centralised logging with correlation ID propagation. This provides a foundation for request tracing across services, but stops short of full observability — metrics and distributed tracing visualisation are planned for V2.

9.1 Logging Architecture

Each microservice publishes structured log events to a central RabbitMQ exchange. A dedicated `log-service` consumes these events asynchronously and persists them to its own PostgreSQL database.

This design decouples logging from business logic — a logging failure never affects the service that produced the event.

```
Service → LogEventProducer → RabbitMQ (TopicExchange)
                                      ↓
                                log-service (consumer)
                                      ↓
                                PostgreSQL (log_event table)
```

9.1.1 RabbitMQ Configuration

The log-service uses a **TopicExchange** with a wildcard routing key (`app.logs.#`). This allows all services to publish to the same exchange using service-specific routing keys, while the log-service consumes everything through a single durable queue.

```
public static final String LOG_EXCHANGE    = "app.logs.exchange";
public static final String LOG_QUEUE       = "app.logs.queue";
public static final String LOG_ROUTING_KEY = "app.logs.#";
```

Both the exchange and queue are **durable** — they survive RabbitMQ restarts. The log-service is self-sufficient: it declares the exchange, queue, and binding on startup, following the infrastructure-as-code principle. No manual setup via the RabbitMQ Management UI is required.

9.1.2 Log Event Structure

Each log event carries the following fields:

Field	Purpose
correlationId	Links all log events belonging to the same request flow
serviceName	Identifies the originating service (e.g. auth-service)
eventType	Business event label (e.g. USER_REGISTERED, LOGIN_FAILED)
level	Severity — INFO, WARN, ERROR
message	Human-readable description
timestamp	When the event occurred
source	HTTP (propagated from gateway) or SYSTEM (internal event)

The `source` field is derived by the log-service consumer: if a `correlationId` is present, the event originated from an HTTP request. If absent, it is an internal system event and a new ID is generated automatically.

9.1.3 Log Event Consumer

The `LogEventListener` consumes messages from the queue asynchronously. Key design decisions:

- **Errors never stop the consumer** — exceptions are caught and logged, but the listener continues processing subsequent messages. A malformed message does not block the queue.
- **Timestamp fallback** — if the producer does not set a timestamp, the consumer uses the arrival time. This handles clock drift between services gracefully.
- **Backward compatibility** — `@JsonIgnoreProperties (ignoreUnknown = true)` on the DTO means the consumer does not break if a producer adds new fields. This is essential in a system where services evolve independently.

9.1.4 Log Storage

Log events are persisted in a dedicated PostgreSQL database (`port 5435`) owned exclusively by the log-service. The `log_event` table has an index on `correlationId` for efficient filtering by request:

```
@Index(name = "idx_log_correlation_id", columnList = "correlationId")
```

Unlike the other services which use `create-drop`, the log-service uses `ddl-auto: update` — logs must persist across restarts. This is the only service where data survives a shutdown.

9.2 Correlation ID Propagation

A correlation ID is a unique identifier assigned to each incoming request and propagated across all services involved in handling that request. It allows all log events from a single user action to be grouped together, making it possible to reconstruct the full request flow from the logs.

9.2.1 Generation at the Gateway

The `CorrelationIdFilter` in the API Gateway is responsible for generating correlation IDs. It runs on every incoming request:

1. Checks if the request already contains `X-Correlation-Id` (useful when the client or an upstream system provides one)
2. If absent, generates a new UUID
3. Stores the value in MDC for logging context
4. Adds it to the response header so clients can reference it
5. Clears MDC after the request completes to prevent thread pollution

9.2.2 Propagation to Internal Services

Each internal service has a `CorrelationIdPropagationFilter` that reads the `X-Correlation-Id` header forwarded by the gateway, stores it in MDC, and adds it to the response header.

The `LogEventProducer` in each service reads the correlation ID from MDC at publish time and attaches it to every log event:

```
String correlationId = MDC.get("correlationId");
event.setCorrelationId(correlationId);
```

This ensures that all log events produced during a request share the same correlation ID, regardless of which service produced them.

9.2.3 Known Limitation: MDC and WebFlux

Both the gateway and internal services use MDC for correlation ID storage. MDC is thread-local — it stores values per thread. In a WebFlux (reactive) environment, a single request may be processed across multiple threads, which means MDC values may not propagate correctly between reactive operators.

The `doFinally` block clears MDC after each request, but this may execute on a different thread than the one that set the value. In practice this works in the current single-host setup, but it is a known fragility. The correct solution for WebFlux is to propagate the correlation ID through the reactive context (`Context`) rather than MDC. This is planned for V2.

9.3 Current Observability Stack

Capability	Tool	Status
Centralised logging	RabbitMQ + log-service	Implemented
Correlation ID tracing	X-Correlation-Id header + MDC	Implemented
Health checks	Spring Actuator	Implemented
Service registry	Eureka dashboard	Implemented
Metrics collection	Prometheus	Not implemented
Metrics visualisation	Grafana	Not implemented
Distributed tracing	Micrometer + Zipkin	Not implemented
Log search UI	Kibana / Grafana Loki	Not implemented

9.4 Observability Gaps and V2 Direction

The current logging setup provides visibility into individual events but lacks the tooling to answer operational questions at scale:

- **No metrics** — there is no way to see request rates, error rates, or latency percentiles in real time. Prometheus with Spring Actuator metrics export is the natural next step.
- **No dashboards** — Grafana connected to Prometheus would provide live visibility into system behaviour without querying the database manually.
- **No distributed tracing UI** — the correlation ID exists in the database but there is no visual tool to show the full request flow across services. Micrometer Tracing with Zipkin would replace the manual correlation ID approach with automatic span recording and a visual timeline.

- **No log search UI** — logs are queryable only via SQL. Grafana Loki or an ELK stack would provide full-text search across all service logs.
- **MDC in WebFlux** — correlation ID propagation via MDC is unreliable in reactive pipelines. Reactive context propagation must replace MDC in V2.

9.5 Lessons Learned

- **Decouple logging from business logic** — publishing to RabbitMQ asynchronously means a logging failure never affects the service. The try-catch in `LogEventProducer` enforces this: logging errors are swallowed, never propagated.
- **Infrastructure as code from day one** — the log-service declares its own RabbitMQ exchange, queue, and binding on startup. No manual setup required. This is the correct approach and should be applied to all infrastructure in V2.
- **Correlation ID is the foundation of observability** — without it, debugging a distributed system means searching through logs from multiple services manually. Even a simple UUID propagated via HTTP header dramatically improves debuggability.
- **MDC and WebFlux do not mix cleanly** — thread-local storage is incompatible with reactive pipelines. This was a known risk when the decision was made and is acceptable for V1, but must be resolved before V2 goes to production.
- **Logging alone is not observability** — logs answer "what happened", metrics answer "how often and how fast", and traces answer "where did time go". All three are needed for production-grade observability. V1 only covers the first.

10 Deployment and Local Infrastructure

The SmartBookingPlatform runs entirely on a local development machine. Infrastructure components are managed via Docker Compose, and all microservices are started and stopped through shell scripts.

This chapter documents the local deployment setup, service ports, and startup behaviour.

10.1 Infrastructure — Docker Compose

All infrastructure dependencies are defined in a single `docker-compose.yaml` file at the root of the project. Running `docker compose up -d` starts all dependencies in detached mode before any Spring Boot service is launched.

The following infrastructure components are managed by Docker:

Container	Image	Port	Purpose
auth-db	postgres:16	5432	Auth Service database
catalog-db	postgres:16	5433	Catalog Service database
booking-db	postgres:16	5434	Booking Service database
log-db	postgres:16	5435	Log Service database
rabbitmq	rabbitmq:3-management	5672 / 15672	Message broker + management UI

Each PostgreSQL instance runs on a different host port, confirming the database-per-service pattern at the infrastructure level — not just separate schemas, but fully isolated database instances.

Docker volumes are used for each database, meaning data persists between Docker restarts. However, because most services use `ddl-auto: create-drop`, the schema and all data are destroyed by Hibernate on every Spring Boot restart regardless of the volume state. The exception is the log-service, which uses `ddl-auto: update` — log data survives application restarts.

To remove all data including volumes:

```
docker compose down -v
```

10.2 Service Ports

Each microservice runs on a fixed port:

Service	Port	Notes
API Gateway	8080	Single entry point for all client requests
Auth Service	8081	Internal only — accessed via gateway
Catalog Service	8082	Internal only — accessed via gateway
Booking Service	8083	Internal only — accessed via gateway
Log Service	8090	Internal only — no REST API exposed
Discovery Service (Eureka)	8761	Service registry dashboard
RabbitMQ Management	15672	Broker management UI

Only the API Gateway (port 8080) and Eureka dashboard (port 8761) are intended for direct access. All business endpoints are accessed exclusively through the gateway.

10.3 Platform Scripts

Two shell scripts at the project root manage the full platform lifecycle.

10.3.1 start-platform.sh

The startup script handles the full platform initialisation sequence:

1. Loads the JWT secret into the environment if not already set
2. Starts all Docker infrastructure in detached mode
3. Waits 5 seconds for infrastructure to be ready
4. Starts all microservices asynchronously using `mvnw spring-boot:run`, each redirecting output to its own `.log` file
5. Polls each service port until it responds or times out after 60 seconds
6. Prints a summary of all running services and their URLs

Services are started in this order:

```
discovery-service   # Must start first     others register with Eureka
auth-service
catalog-service
booking-service
log-service
api-gateway         # Started last     routes to all other services
```

The Discovery Service must be available before other services start, as they register with Eureka on startup. The API Gateway is started last because it depends on all services being registered.

10.3.2 stop-platform.sh

The stop script terminates all running services and infrastructure:

```
taskkill //F //IM java.exe   # Stops all Java processes (Windows)
docker compose down           # Stops and removes containers
```

`taskkill` stops all Java processes at once rather than individually per service. This is a deliberate simplification — acceptable in a local development environment where no other Java processes are expected to be running.

Note that `docker compose down` without `-v` preserves volumes. This only matters for the log-service, where data survives between restarts due to `ddl-auto: update`.

10.4 Startup Observations

- **Cold start time** — the full platform takes approximately 60-90 seconds to be fully operational from a cold start, dominated by JVM startup time across 6 services.
- **Windows-specific script** — `taskkill` is a Windows command. The stop script requires adaptation for Linux or macOS (`pkill -f spring-boot` or similar).
- **No health check on RabbitMQ** — the startup script waits 5 seconds for infrastructure but does not verify that RabbitMQ is actually accepting connections before starting services. If RabbitMQ is slow to start, services may fail their initial connection attempt and rely on Spring's retry mechanism.
- **Log files** — each service writes its output to a `.log` file in its own directory (e.g. `auth-service/auth-servic` These are useful for debugging startup failures when running the platform headlessly.

10.5 Lessons Learned

- **Start order matters** — in a microservices system, services that depend on shared infrastructure (Eureka, RabbitMQ, PostgreSQL) must not start before that infrastructure is ready. A fixed sleep is a fragile solution — a proper health check loop per dependency is more reliable and is the standard approach in Kubernetes with readiness probes.
- **Automate everything from day one** — having a single command to start and stop the entire platform eliminates friction and reduces the chance of forgetting to start a dependency. This is the local equivalent of what Kubernetes does in production.

- **Separate infrastructure from application** — Docker manages infrastructure, Maven manages applications. Keeping these separate makes it easy to restart a single service without touching the database or broker.
- **Windows scripts need adaptation** — `taskkill` is not portable. In V2 with Docker and Kubernetes, this problem disappears — services are stopped via `kubectl` or `docker compose` regardless of OS.

11 Limitations and V2 Roadmap

V1 of the SmartBookingPlatform successfully demonstrates a microservices architecture with service discovery, JWT-based security, resilience patterns, centralised logging, and load testing. However, several limitations were deliberately accepted in favour of simplicity and learning velocity.

This chapter consolidates those limitations and maps each one to a concrete V2 improvement.

11.1 Known Limitations

Limitation	Context
No role-based authorization	JWT contains roles but no enforcement exists. All authenticated requests are treated equally.
No integration tests	All tests mock dependencies. No end-to-end flow is validated across real services and databases.
MDC and WebFilter issues	API Gateway uses MDC (thread-local) in a reactive pipeline — correlation ID may not propagate correctly across threads. Internal services (Auth, Catalog, Booking) use `WebFilter` incorrectly — the correct interface for a servlet-based stack is `OncePerRequestFilter`.
Schema management	All services use `create-drop` except log-service. Data is destroyed on every restart.
JWT secret via file	Secret is shared via `.env` file. Vault was evaluated but removed due to operational complexity of local token management.
Infrastructure-dependent tests	Tests using `@SpringBootTest` fail without active PostgreSQL and RabbitMQ.
No metrics or dashboards	No Prometheus, no Grafana. System behaviour under load is only visible via k6 output.
No distributed tracing UI	Correlation IDs exist in the database but there is no visual tool to reconstruct request flows.
No concurrency protection	Booking conflict detection relies on application-level checks. No optimistic locking or idempotency keys.
Synchronous coupling	Booking Service is completely blocked when Catalog Service is unavailable.

11.2 V2 Roadmap

11.2.1 Messaging — Kafka

RabbitMQ will be retained for logging. Apache Kafka will be introduced for event-driven data replication between services.

The Catalog Service will publish events such as `ResourceCreated`, `ResourceUpdated`, and `ResourceDeactivated`. The Booking Service will consume these events to maintain a local copy of stable resource data (name, price, duration), removing the need to call the Catalog Service synchronously on every booking request.

Real-time availability validation will remain synchronous — a hybrid model as discussed in Chapter 5.

A decision will also be made on whether to consolidate RabbitMQ and Kafka into a single broker or keep them separate for different concerns.

11.2.2 Observability — Metrics and Tracing

Three improvements are planned:

- **Prometheus + Spring Actuator** — expose metrics endpoints per service and scrape them with Prometheus. Metrics include request rate, error rate, latency percentiles, and JVM health.
- **Grafana** — dashboards for live system visibility, replacing manual k6 output analysis with persistent monitoring.
- **Micrometer Tracing + Zipkin** — replace the manual correlation ID approach with automatic distributed tracing. Each request will produce spans across all services, visible in a Zipkin UI as a timeline. This supersedes the current `X-Correlation-Id` header propagation.

The MDC and WebFilter issues identified in Chapter 9 will also be resolved: gateway correlation ID propagation will use reactive context instead of MDC, and internal services will use `OncePerRequestFilter` instead of `WebFilter`.

11.2.3 Security — Role-Based Authorization

Role-based authorization will be enforced at the gateway level for coarse-grained rules:

- Only `PROVIDER` can create or delete offers and resources
- Only `CLIENT` can create bookings
- Only `ADMIN` can manage users

Fine-grained rules (e.g. a provider can only delete their own offers) will be enforced inside each service.

HashiCorp Vault will be re-evaluated for secret management. The main challenge in V1 was the ephemeral token model in local development — tokens expire and require manual renewal, breaking the development workflow. Alternatives to evaluate include Vault with a long-lived token for development, or Spring Cloud Config Server as a simpler alternative for managing shared secrets across services.

11.2.4 Testing — Integration and Contract Tests

Two testing gaps will be addressed:

- **Integration tests** — end-to-end tests covering the full booking flow across real services and databases. These will run against a Docker Compose environment and be separated into a dedicated Maven profile so they do not block the standard unit test cycle.
- **Contract tests with Pact** — consumer-driven contract tests between the Booking Service and Catalog Service. If the resource response DTO changes in Catalog, the test suite will fail at build time rather than at runtime.

A CI/CD pipeline will also be introduced — likely GitHub Actions — running unit tests on every push and integration tests on pull requests.

11.2.5 Deployment — Kubernetes and Flyway

- **Kubernetes** — each service will be containerised and deployed as a Kubernetes workload. This enables independent scaling, rolling deployments, and self-healing.
- **Flyway** — `create-drop` will be replaced with versioned schema migrations across all services. This is a prerequisite for any production deployment.

11.2.6 Concurrency — Booking Integrity

The current booking conflict detection relies on an application-level overlap check. Under concurrent load, two requests arriving simultaneously could both pass the check before either is persisted — a classic race condition.

Three mechanisms will be evaluated for V2:

- **Optimistic locking** — using `@Version` on the booking entity to detect concurrent modifications at the database level
- **Unique constraint** — adding a database-level constraint to prevent overlapping bookings for the same resource
- **Idempotency key** — clients provide a unique key per request, preventing duplicate bookings from retries or network issues

Load testing with k6 will be extended in V2 to simulate true concurrent booking attempts and validate that the chosen mechanism prevents double-booking under real concurrency.

11.3 V2 Summary

Area	Improvement	Resolves
Messaging	Kafka hybrid model	Synchronous coupling
Observability	Prometheus, Grafana, Zipkin	No metrics or tracing
Observability	Reactive context propagation	MDC and WebFilter issues
Security	Role-based authorization	No access control
Security	Vault re-evaluation	JWT secret in .env file
Testing	Integration + contract tests	No end-to-end coverage
Testing	CI/CD pipeline	Manual test execution
Deployment	Kubernetes	No container orchestration
Deployment	Flyway	Schema destroy on restart
Concurrency	Optimistic locking / idempotency	Race condition risk

12 Conclusion

The SmartBookingPlatform V1 set out with a clear goal: to build a microservices-based backend system using modern Java and Spring technologies, and to document the architectural decisions, trade-offs, and lessons learned along the way.

What Was Built

The platform is composed of six independent services — API Gateway, Auth Service, Catalog Service, Booking Service, Log Service, and Discovery Service — each with a clear responsibility, its own database, and a well-defined contract with the rest of the system.

JWT-based stateless authentication, resilience patterns, centralised logging with correlation ID propagation, and load testing with k6 were all implemented, tested, and documented. The test suite closes at 72 passing tests across 17 test classes and three services.

What Was Learned

The most valuable outcome of this project is not the code — it is the ability to explain every decision.

Why microservices over a modular monolith for a solo developer. Why synchronous communication for availability validation but not for stable resource data. Why centralising security at the gateway reduces duplication but introduces implicit trust. Why a 409 in a booking system is not a failure but a business rule enforced correctly.

These are not textbook answers. They are the product of building something real, hitting real problems, and thinking carefully about the trade-offs.

What Comes Next

V2 will go further — Kafka, Kubernetes, Prometheus, Grafana, distributed tracing, role-based authorization, integration tests, and proper schema management with Flyway.

But V2 starts from a strong foundation: a system that works, a test suite that passes, and a document that captures everything worth remembering.

Final Thought

The goal was never to build the perfect system.
The goal was to build something worth learning from.